ML MODEL DEPLOYMENT STRATEGIES AND MODEL SERVING USING FASTAPI

CLA 4 — Observation Report

Dataset: Titanic — Machine Learning from Disaster (Kaggle, labelled train.csv)

Objective: To train and evaluate a classification model, save the complete preprocessing and prediction pipeline, demonstrate online, batch and streaming inference using unseen passenger records, and expose predictions through a REST API implemented with FastAPI.

1. Introduction

Machine learning deployment involves making a trained model available for predictions on data that was not used during training. Model training is only one part of a machine learning workflow. After training, a model must be saved, loaded when required, supplied with appropriately formatted input data, and made accessible to other programs or users. This assignment demonstrates several ways of performing inference and introduces API-based model serving.

The Titanic dataset contains passenger attributes and the target variable Survived. The model uses passenger class (Pclass), age (Age), number of siblings or spouses aboard (SibSp), number of parents or children aboard (Parch), fare (Fare), sex (Sex), and port of embarkation (Embarked). Passenger identifiers and descriptive fields that are not used by the model are excluded from the feature set.

2. Model Training and Evaluation

The labelled Kaggle train.csv file is loaded into a pandas DataFrame. Rows without a target label are excluded. The labelled data is divided into training and testing portions using an 80:20 split and a fixed random seed. Stratification is used so that the distribution of the survival classes is represented in both portions. The test portion is held out from model fitting and is used to estimate generalisation performance.

Preprocessing is included inside a scikit-learn Pipeline. Missing numeric values are replaced with the median, and numeric features are standardised. Missing categorical values are replaced with the most frequent category, while OneHotEncoder converts categorical values into numeric indicator features. A RandomForestClassifier is then fitted to the processed training data. Combining preprocessing and classification into a single pipeline helps ensure that the same transformations are applied during training and prediction.

Accuracy is calculated by comparing predictions on the held-out test data with the known target labels. Accuracy is the proportion of test records for which the predicted survival value matches the actual value. The exact score is dataset- and split-dependent, so the notebook prints the measured result when it is executed. The complete fitted pipeline is saved using Joblib as model.pkl; this file contains both preprocessing steps and the trained classifier.

3. Online Inference

Online inference produces a prediction for one record when a result is requested. In this assignment, the saved model.pkl is loaded and one passenger from the held-out test portion is selected. The selected row is passed to the pipeline, which performs the required preprocessing and returns a predicted survival class. The output is displayed as a numeric class and a readable label: Survived or Did not survive. The actual held-out label may also be shown for comparison, but it is not supplied to the model as an input feature.

Online inference is useful when an application needs a result for an individual request, such as a form submission or a single passenger query. It typically prioritises returning a response promptly rather than waiting for a large collection of records to accumulate.

4. Batch Inference

Batch inference processes a group of records together. The project selects 20 records from the held-out test portion and calls the model to generate predictions for the complete group. The results are combined with the passenger features and, for evaluation, the corresponding actual labels. The output is saved as batch_predictions.csv so that it can be inspected, reused, or submitted as an assignment artifact.

Batch inference is appropriate when many predictions are needed at once and an immediate response for every individual record is not necessary. Processing a group together can reduce repeated call overhead and is often convenient for scheduled reports, offline analysis, and bulk scoring tasks. The batch file records each prediction next to the input features, making the output easy to review.

5. Streaming Inference

Streaming inference processes incoming records sequentially as they arrive. The assignment simulates this behaviour using 10 held-out passenger records. Each record is passed to the same loaded model individually, its predicted class is printed, and the program waits one second before processing the next record. This delay is included to make the sequence of incoming records visible during the demonstration.

A simulated stream is not the same as a production event-streaming platform. It is a simple demonstration of the one-record-at-a-time pattern. In a real system, records might arrive from an application, message queue, sensor, or event service. Streaming can be useful when each new event should be processed without waiting for a whole batch to be collected.

6. Model Serving with FastAPI

FastAPI exposes the saved machine learning pipeline through HTTP endpoints so that another application can request predictions. The application loads model.pkl and defines a Pydantic request schema for passenger details. The schema validates fields such as passenger class, age, number of relatives aboard, fare, sex, and embarkation port. Invalid values, such as a passenger class outside the supported range, are rejected by request validation instead of being passed silently to the model.

The GET /health endpoint reports whether the API is responding and whether the model was loaded. The POST /predict endpoint accepts a passenger record, converts the validated request into a DataFrame with the same feature names expected by the pipeline, runs prediction, and returns a JSON response containing the numeric prediction and readable label. FastAPI also generates interactive documentation at /docs, where endpoints can be exercised manually.

The project tests the API with three requests: a health check, a valid prediction request, and an invalid prediction request. The expected outcomes are HTTP 200 for the health check, HTTP 200 for a valid prediction, and HTTP 422 for a request that violates the Pydantic schema. A separate Python client is also provided to call the running service using requests. In Google Colab, the notebook's TestClient checks the endpoints in-process; starting Uvicorn is an optional step for a live server demonstration.

7. Observations and Answers

7.1 How does Online Inference differ from Batch Inference?

Online inference predicts one record at a time when a result is requested. It is suitable for interactive systems where a user or application needs an individual prediction promptly. Batch inference predicts a collection of records in one operation. It is suitable for bulk scoring or scheduled processing when results can be produced together. The main difference is the unit and timing of work: a single request in online inference versus a group of records in batch inference.

7.2 What is the difference between Streaming Inference and Batch Inference?

Streaming inference processes each incoming record as it arrives, usually continuing over time. The current project simulates a stream by processing 10 records sequentially with a one-second pause between records. Batch inference first selects a fixed group of records and then predicts the group together. Streaming is event-oriented and incremental, whereas batch inference is group-oriented and typically runs after a collection has been prepared.

7.3 Why is FastAPI used for model serving?

FastAPI provides a convenient way to make the trained model accessible through REST-style HTTP endpoints. It supports typed request definitions and Pydantic validation, produces clear error responses for invalid input, and automatically creates interactive API documentation at /docs. This separates the model from the client application: a web page, script, or other service can request predictions without needing to load the model directly.

7.4 Why should the same trained model be reused for all inference strategies?

The same saved pipeline should be reused so that online, batch, streaming, and API predictions use identical fitted preprocessing and classifier parameters. If a new model were trained separately for each method, differences in the random split, learned preprocessing values, or model parameters could cause inconsistent predictions. Saving and reloading model.pkl also demonstrates the normal deployment pattern: train and evaluate once, then serve the fitted artifact for inference.

8. Files Generated and Expected Results

Artifact	Purpose
model.pkl	Saved fitted preprocessing and classification pipeline.
heldout_passengers.csv	Held-out passenger records used for inference demonstrations and label comparison.
batch_predictions.csv	Predictions for 20 held-out passengers, along with their features and actual labels.
app.py	FastAPI application containing /health and /predict endpoints.
client.py	Python client that calls the running API.
requirements.txt	Python package dependencies needed to run the project.

When the notebook is run successfully, the console displays the measured test accuracy, the online prediction, the 20-row batch result, the sequence of 10 streaming predictions, and the results of the FastAPI endpoint tests. Screenshots of these actual outputs should be captured during execution and included with the final submission.

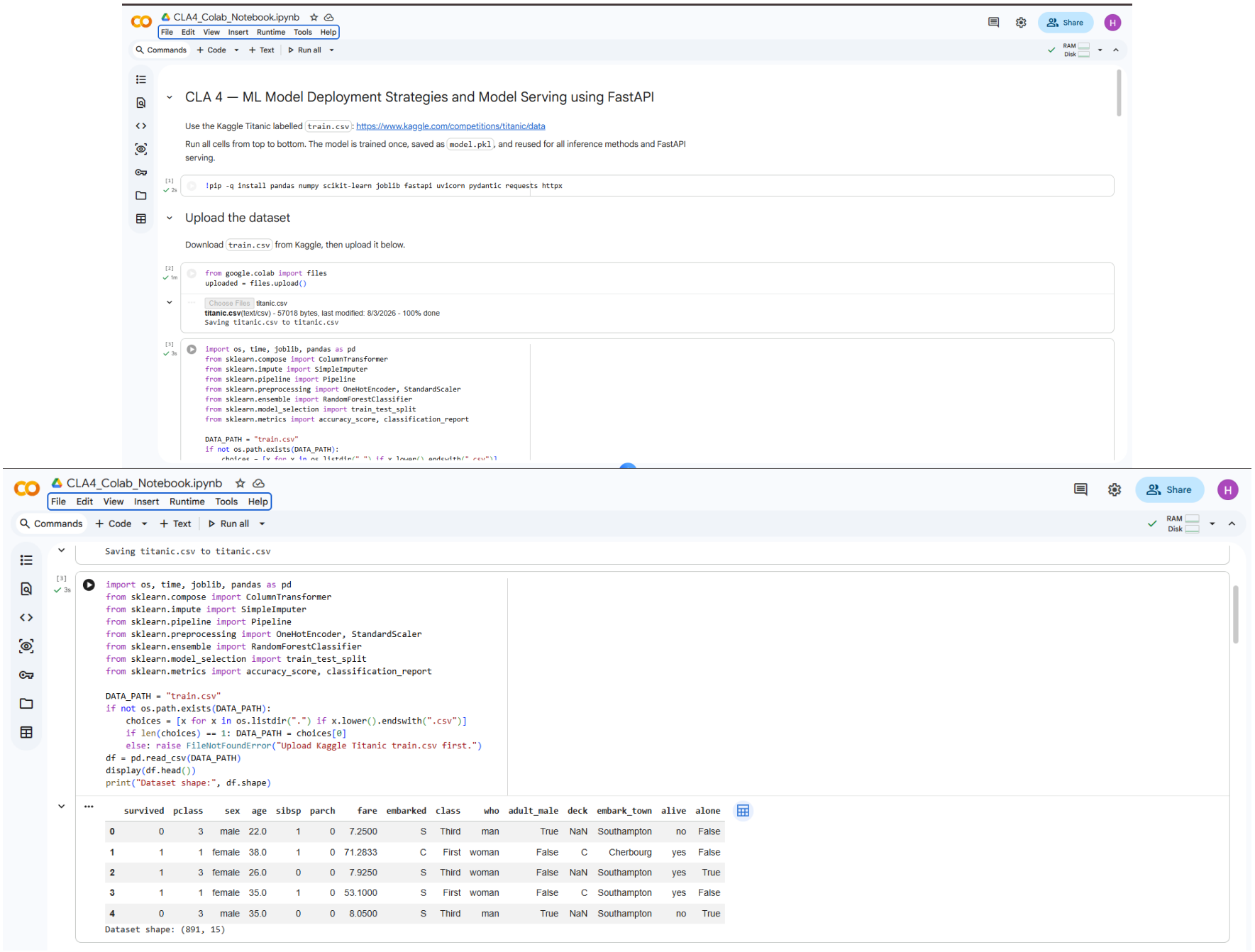
9. Conclusion

This assignment demonstrates the path from model development to basic deployment. A scikit-learn pipeline is trained on the labelled Titanic dataset, evaluated on a held-out test set, and saved as model.pkl. The same artifact is then reused for online, batch, and streaming inference. Finally, FastAPI makes predictions available through validated HTTP requests. Together, these steps illustrate why a reproducible saved pipeline, clear input validation, and consistent inference behaviour are important when making a machine learning model available to other applications.

Reproducibility note: The numerical accuracy and passenger predictions are intentionally not hard-coded in this report. They should be copied from the actual Colab execution after running the notebook with the official Kaggle train.csv dataset.

Colab Notebook — Dataset preview and loading

Execution evidence captured from the user's Google Colab notebook.

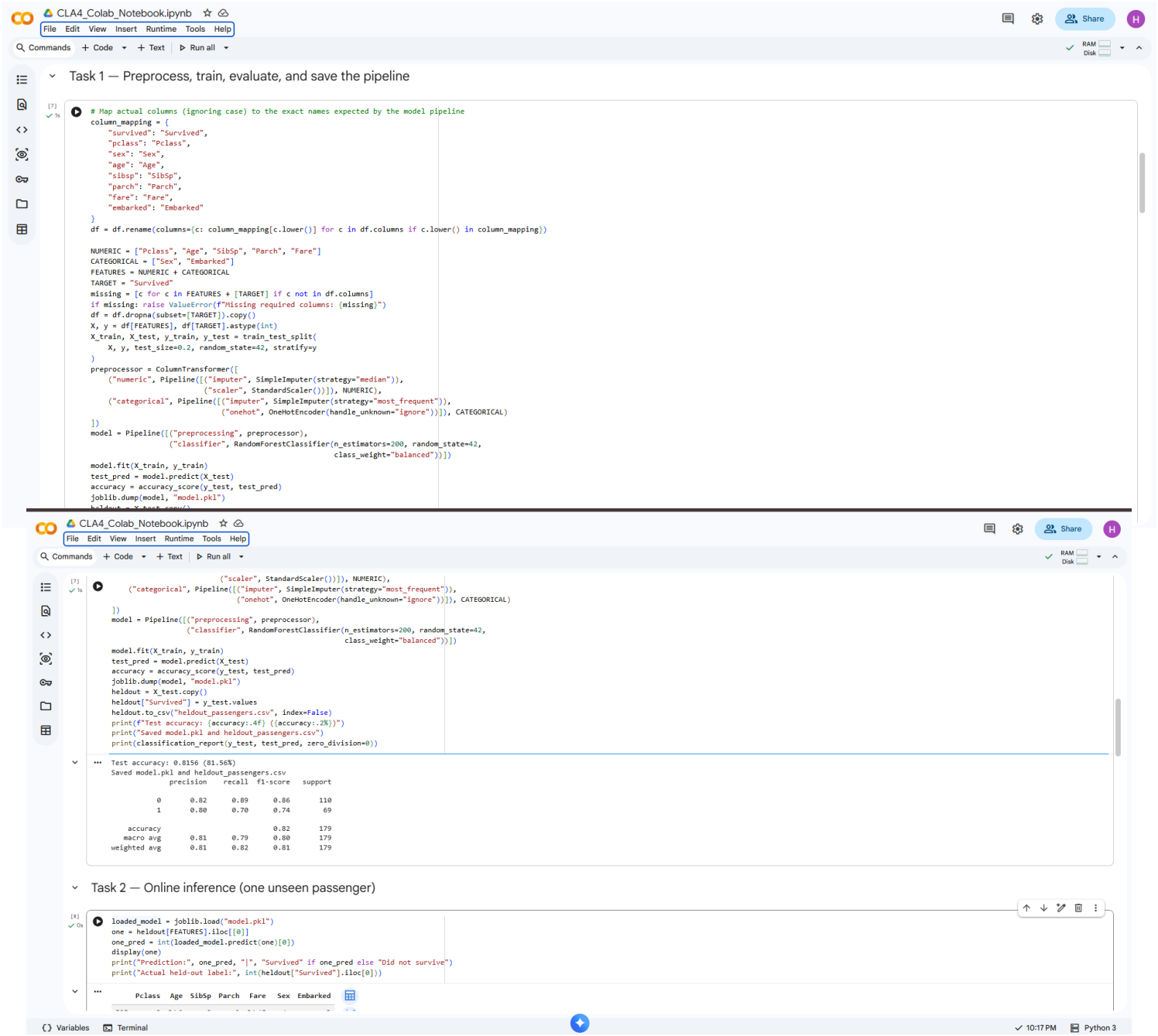


	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Dataset shape: (891, 15)

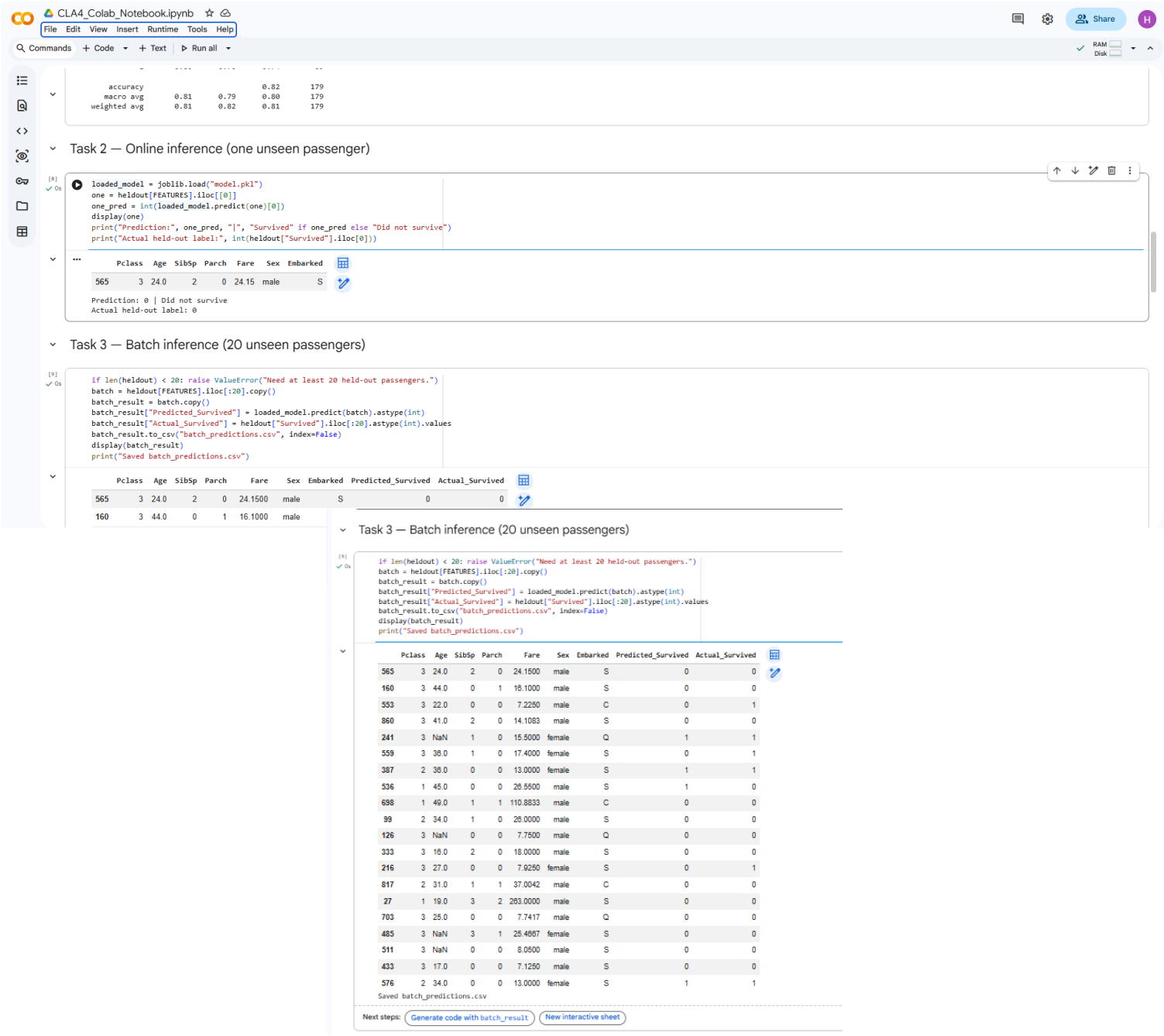
Task 2 — Online inference, accuracy, and classification report

Execution evidence captured from the user's Google Colab notebook.



Task 3 — Batch inference for 20 passengers

Execution evidence captured from the user's Google Colab notebook.



Task 6 — Evaluate endpoint tests for valid and invalid requests

Execution evidence captured from the user's Google Colab notebook.

Task 4 — Streaming inference (10 records, one-second delay)

[10] ✓ 9s

```
stream_predictions = []
for i in range(10):
    row = heldout[FEATURES].iloc[[i]]
    pred = int(loaded_model.predict(row)[0])
    stream_predictions.append(pred)
    print(f"Incoming record {i+1}/10: predicted Survived={pred}")
    if i < 9: time.sleep(1)
```

```
Incoming record 1/10: predicted Survived=0
Incoming record 2/10: predicted Survived=0
Incoming record 3/10: predicted Survived=0
Incoming record 4/10: predicted Survived=0
Incoming record 5/10: predicted Survived=1
Incoming record 6/10: predicted Survived=0
Incoming record 7/10: predicted Survived=1
Incoming record 8/10: predicted Survived=1
Incoming record 9/10: predicted Survived=0
Incoming record 10/10: predicted Survived=0
```

CLA4_Colab_Notebook.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text ▷ Run all

RAM Disk

[11] ✓ 0s

```
valid = {"Pclass":3,"Sex":"female","Age":22,"SibSp":1,"Parch":0,"Fare":7.25,"Embarked":"S"}
valid_response = api_client.post("/predict", json=valid)
print("Valid POST /predict:", valid_response.status_code, valid_response.json())
invalid = {"Pclass":9,"Sex":"female","Age":22,"SibSp":1,"Parch":0,"Fare":7.25,"Embarked":"S"}
invalid_response = api_client.post("/predict", json=invalid)
print("Invalid POST /predict:", invalid_response.status_code, invalid_response.json())
assert health_response.status_code == 200
assert valid_response.status_code == 200
assert invalid_response.status_code == 422
print("All API endpoint tests passed.")
```

```
GET /health: 200 {'status': 'ok', 'model_loaded': True}
Valid POST /predict: 200 {'prediction': 1, 'label': 'Survived'}
Invalid POST /predict: 422 {'detail': [{'type': 'less_than_equal', 'loc': ['body', 'Pclass'], 'msg': 'Input should be less than or equal to 3', 'input': 9, 'ctx': {'le': 3}}]}
All API endpoint tests passed.
```

Optional: launch the live API server

[12] ✓ 0s

```
# To use Swagger UI, save the app to app.py (provided in the project ZIP), then run:
# !uvicorn app:app --host 0.0.0.0 --port 8000
# In Colab, a local server is not automatically public; a tunnel is needed for browser access.
```

Task 6 — Observations

1. **Online vs Batch:** Online predicts one record on demand; batch predicts many records together.
2. **Streaming vs Batch:** Streaming handles records one at a time as they arrive; batch processes a fixed collection together.
3. **Why FastAPI:** It exposes predictions via HTTP, validates data with Pydantic, and provides interactive Swagger documentation at `/docs`.
4. **Why reuse the model:** The same saved pipeline ensures consistent preprocessing and predictions across all inference methods.

[13] ✓ 0s

```
from google.colab import files
import zipfile
with zipfile.ZipFile("CLA4_generated_outputs.zip", "w", zipfile.ZIP_DEFLATED) as z:
    for filename in ["model.pkl", "heldout_passengers.csv", "batch_predictions.csv"]:
        if os.path.exists(filename): z.write(filename)
files.download("CLA4_generated_outputs.zip")
```

Variables Terminal ✓ 10:17 PM Python 3